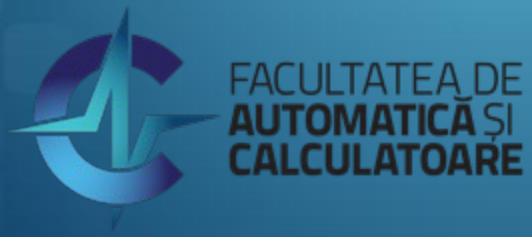


Algoritmi Paraleli și Distribuiți
Comunicarea prin mesaje. Complexitatea algoritmilor distribuiți

Radu-Ioan CIOBANU
radu.ciobanu@upb.ro





Programarea paralelă

Multithreaded Programming



Programarea paralelă (2)



Programarea paralelă (3)



Feedback

https://forms.office.com/r/MxvEiuLFFW



1. Comunicarea prin mesaje

Transmiterea de mesaje

- Îndeplinește cele două funcții:
  - **comunicarea** - datele ajung de la transmiţător la receptor
  - **sincronizarea** - un mesaj nu poate fi recepţionat înainte de a fi transmis
- **Canalele** sunt obiectele puse în comun de procese
- **Programe distribuite**

Comunicare asincrona prin mesaje

Comunicarea asincronă prin mesaje

Modelul

- Canalele păstrează ordinea mesajelor
- Canalele nu pierd mesajele
- **canalele** sunt cozi FIFO de mesaje, de capacitate nelimitata
- operaţia **send** nu este blocantă
- operaţia **receive** este blocantă

Declaraţie – forma generală

- **chan** nume_canal (tip1 id1,..., tipn idn)

Grup indexat de canale

- **chan** rezultat [1:n](int)

Operaţii

- **send** nume_canal (expresie$_1$,..., expresie$_n$)
- **receive** nume_canal (variabila$_1$,..., variabila$_n$)
- **empty** (nume_canal)

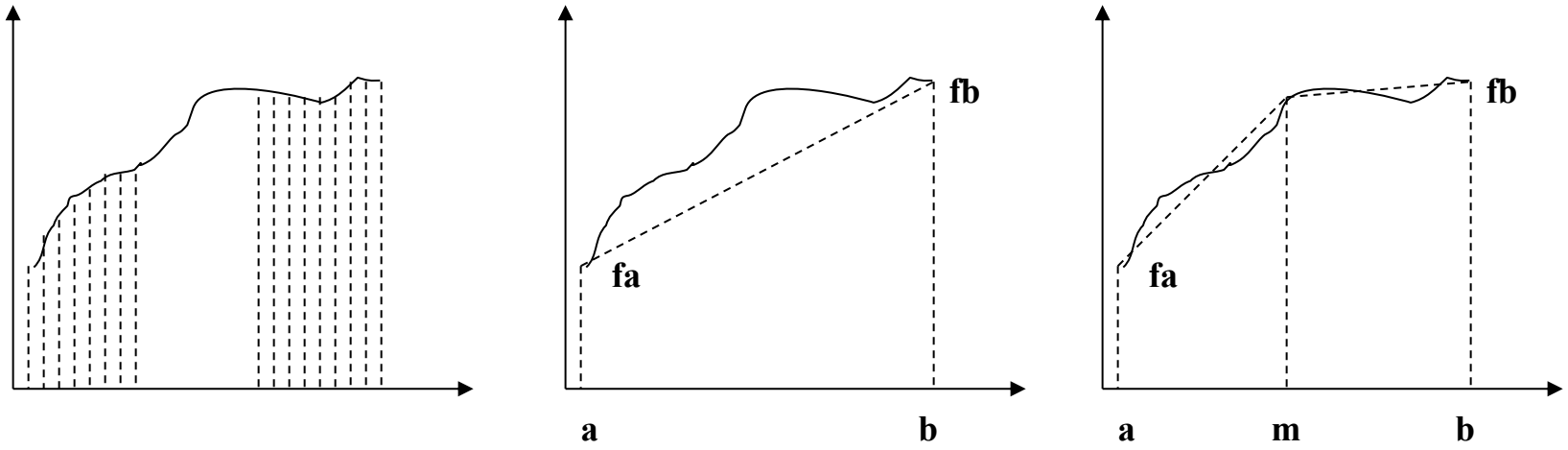
Exemplu - Filtru

```
chan   input (char),
       output (char [1:Maxl]);

process car_linii{
  char line[1:Maxl]; int i=1;
  while (true) {
    receive input (line[i]);
    while (line[i] <> CR and i < Maxl) {
      /* line[1:i] = ultimele i caractere introduse*/
      i = i + 1;
      receive input (line[i]);
    }
    send output (line);
    i = 1;
  }
}
```

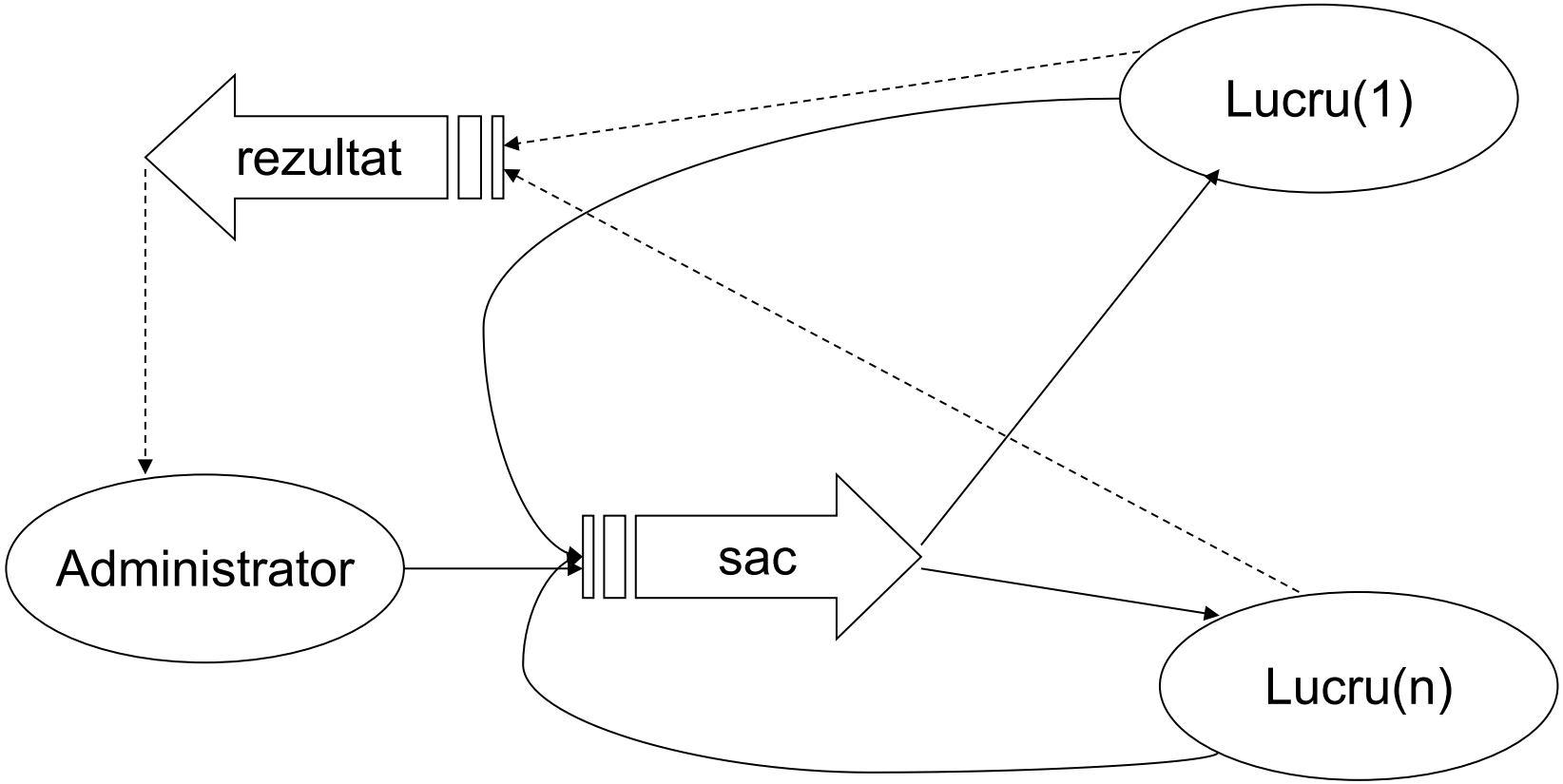
Modelul Replicated Workers

Integrare numerica



- Stanga – varianta secventiala
  - calcul iterativ - la fiecare pas se dubleaza numarul de subintervale
  - se termina cand diferenta intre doua valori succesive este acceptabila
- Dreapta – varianta distribuita
  - un proces primeste un subinterval [a,b] si aria corespunzatoare
  - imparte intervalul in doua sub-intervale [a.m], [m,b] si calculeaza ariile
  - compara suma celor doua arii cu aria primita
  - continua calculul separat pe cele doua subintervale, daca este cazul

Replicated workers



Exemplu - Replicated Workers (2)

```
chan sac (real a, b, fa, fb, aria);
chan rezultat (real a, b, aria);

process Administrator{
  real xl, xr, yl, yr, a, b, aria, total: real;
  /* alte variabile marcare intervale terminate */

  yl = f(xl); yr = f(xr);
  aria = (yl + yr) * (xr - xl) / 2;
  send sac(xl, xr, yl, yr, aria);

  while (nu s-a calculat toata aria) {
    receive rezultat (a, b, aria);
    total = total + aria;
    marchează intervalul [a, b] terminat;
  }
}
```

Exemplu - Replicated Workers (3)

```
process Lucru[i=1 to n]{
   real a, b, m, ya, yb, ym;
   real ariaSt, ariaDr, ariaTot, dif;
   while (true) {
      receive sac (a, b, ya, yb, ariaTot);
      m = (a + b) / 2;
      ym = f(m);
      calculează ariaSt și ariaDr;
      dif = ariaTot - ariaSt - ariaDr;
      if (dif mic) send rezultat (a, b, ariaTot);
      else { send sac (a, m, ya, ym, ariaSt);
             send sac (m, b, ym, yb, ariaDr);
      }
   }
}
```

Comunicare sincrona prin mesaje

Comunicarea sincrona prin mesaje

Modelul

Caracteristici similare cu comunicarea asincrona

Diferenta: si operaţia **send** **este** blocantă – transmitatorul se blocheaza pana cand mesajul este receptionat

Operaţii

synch_send nume_canal (expresie$_1$,..., expresie$_n$)

receive nume_canal (variabila$_1$,..., variabila$_n$)

empty (nume_canal)

Neajunsuri

reduce concurenta

Exemplu – comunicare producator / consumator

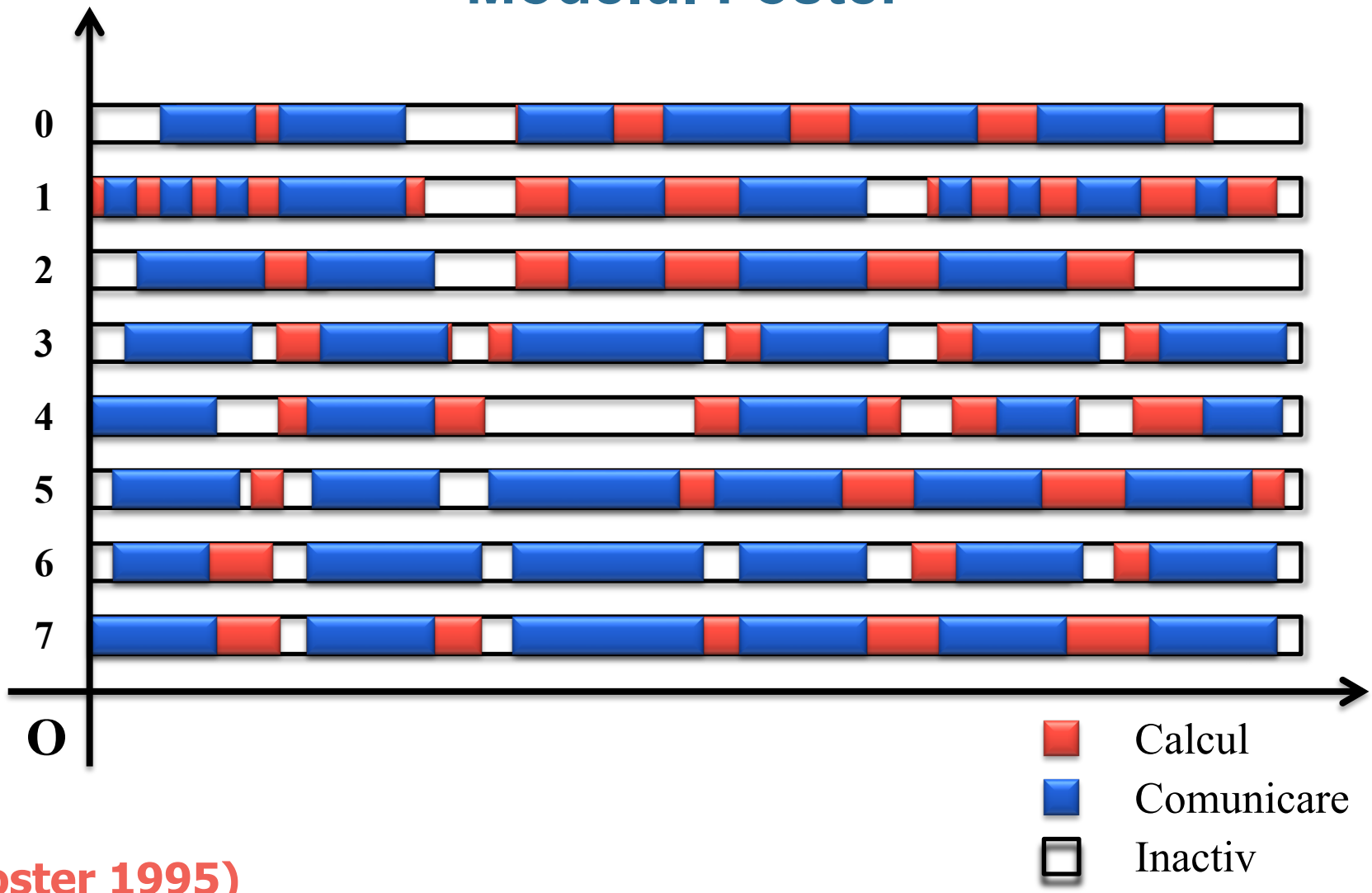
```
chan prodcons (real);

process prod {
    real date[n];
    for [i = 1 to n] {
        calculeaza date[i];
        synch_send prodcons (date[i]);     /* intarzie transmitatorul */
    }                                      /* daca receptorul este mai lent */
}
process cons {
    real buf[n];
    for [i = 1 to n] {
        receive prodcons (buf[i]);
        prelucreaza datele primite;
    }
}
```

2. Complexitatea algoritmilor distribuiți

Modelul Foster

Modelul Foster



(Foster 1995)

Timpul total de execuţie

- Definiţie
  - Timpul scurs de la începerea execuţiei primului proces până la terminarea execuţiei ultimului proces.

$$T = f\ (\ N,\ P,\ U, \ldots\ )$$

$$= T^{j}_{comp} + T^{j}_{commun} + T^{j}_{idle}$$

$$= \left(\frac{1}{P}\right) * \left(\sum_{i=0}^{P-1} T^{i}_{comp} + \sum_{i=0}^{P-1} T^{i}_{commun} + \sum_{i=0}^{P-1} T^{i}_{idle}\right)$$

$$= \left(\frac{1}{P}\right) * \left(T_{comp} + T_{commun} + T_{idle}\right)$$

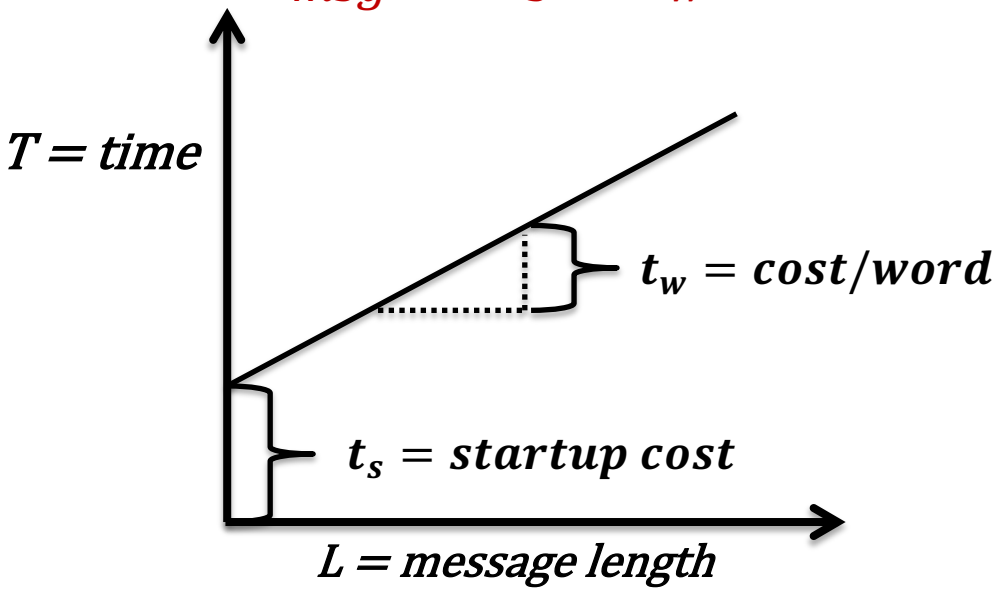
Timpul de calcul - T_{comp}

- Depinde de:
  - dimensiunea problemei N
  - numărul de task-uri sau procesoare
  - caracteristicile procesoarelor şi memoriei
- Exemplu:
  - scalarea dimensiunii problemei sau a numărului de procesoare poate modifica performanța cache-ului sau eficacitatea pipelining-ului procesorului

Timpul de comunicare – T_{commun} (1)

- Timpul petrecut cu transmiterea - recepţia de date
- Tipuri:
  - inter-procesor
  - intra-procesor
- Presupunere: au costuri comparabile

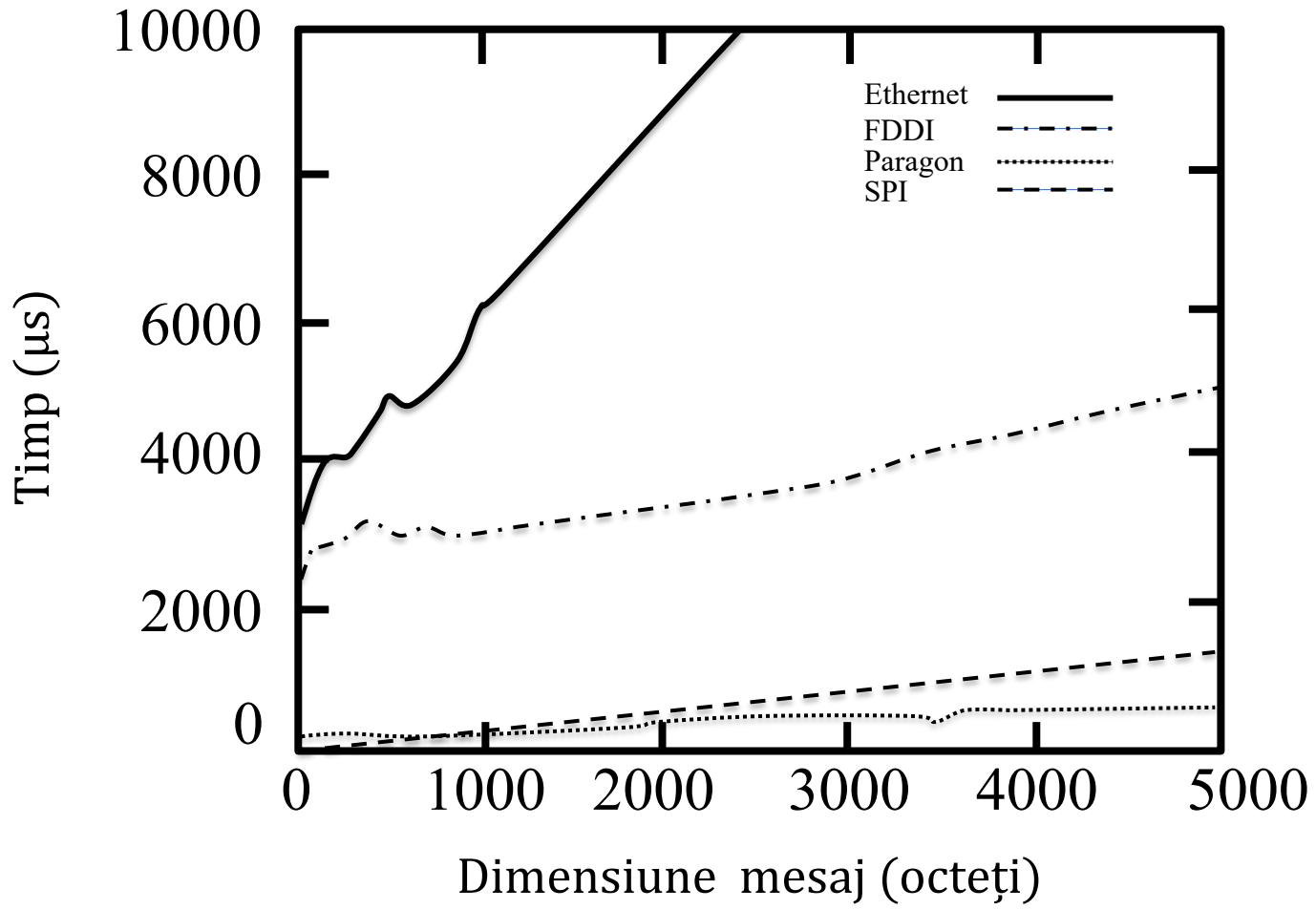
$$T_{msg} = t_s + t_w L$$



Machine	t_s (µs)	t_w(µs)
IBM SP2	40	0.11
Intel DELTA	77	0.54
Intel Paragon	121	0.07
Meiko CS-2	87	0.08
nCUBE-2	154	2.4
Thinking Machines CM-5	82	0.44
Workstations on Ethernet	1500	5.0
Workstations on FDDI	1150	1.1

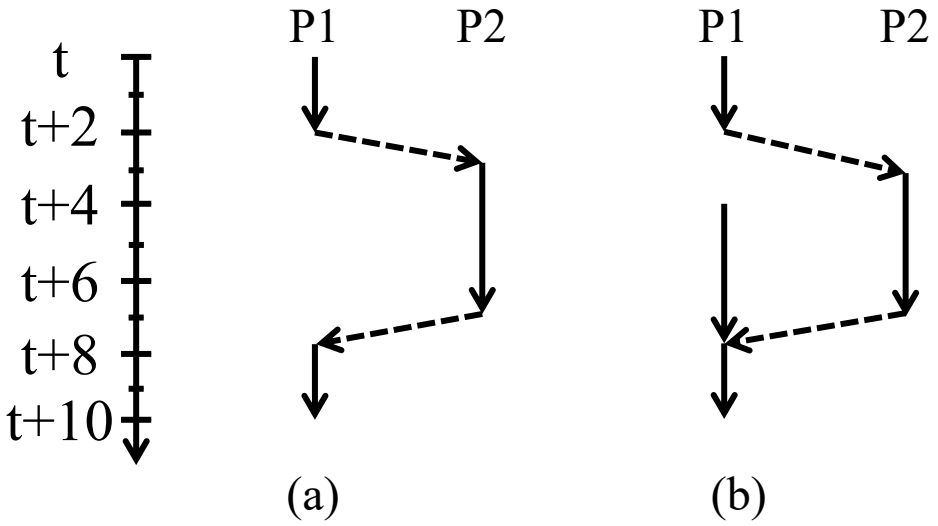
Timpul de comunicare – T_{commun} (1)

- Măsurători experimentale: timp dus-întors pentru mesaje (RTT)



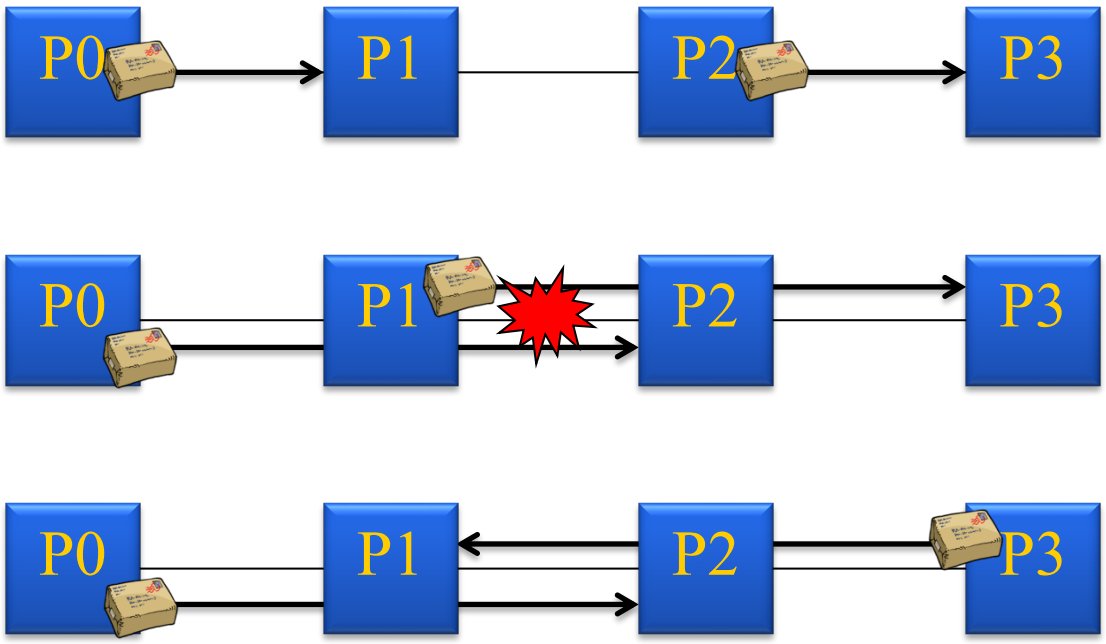
Timpul idle - T_{idle}

- Datorită:
  - lipsei calculelor → *load balancing*
  - lipsei datelor → *overlapping computation and communication*



Model revizuit pentru cost comunicare

- Model comunicare simplu: $T_{msg} = t_s + t_w L$
- Competiţie pentru bandă: $T_{msg\text{-}b} = t_s + t_w SL$
- S = număr de procesoare care comunică concurent pe acelaşi canal (și în același sens)



Exemplu – Sequential Floyd

- $I[i,j]_0 = 0$, daca i == j
- $I[i,j]_0 = cost(i,j)$, dacă există muchie și i != j
- $I[i,j]_0 = MAX$, altfel

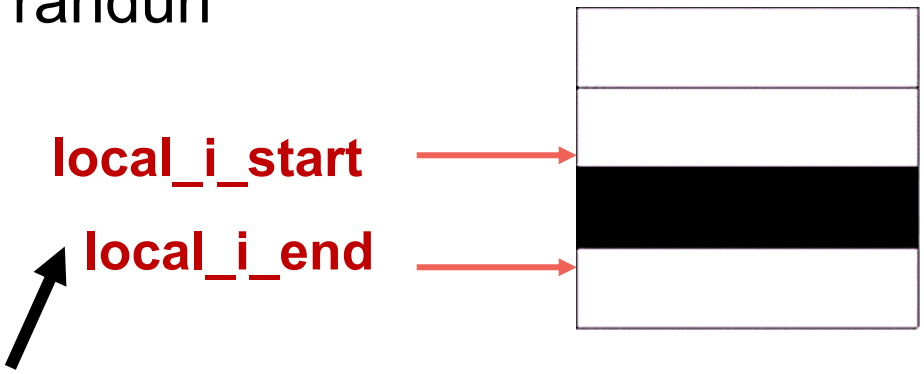
```
for [k = 0 to N - 1]
  for [i = 0 to N - 1]
    for [j = 0 to N - 1]
      I[i,j]k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k)
```

Durata unei iterații

$$T = t_c N^3$$

Exemplu – Parallel Floyd 1 (1)

- Bazat pe o descompunere uni-dimensională pe rânduri a matricei I (algoritmul poate folosi cel mult P procesoare, P<=N)
- Fiecare task are unul sau mai multe rânduri adiacente din I şi răspunde de calculul acestor rânduri




```
for [k = 0 to N-1]
   for [i = local_i_start to local_i_end]
     for [j = 0 to N-1]
     I[i,j]k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k)
```

Exemplu – Parallel Floyd 1 (2)

- În pasul k, procesele au nevoie de linia k din matricea I.
- Procesorul cu această linie o difuzează tuturor în log P paşi folosind o structură arborescentă.
- Deoarece fiecare mesaj are N cuvinte, timpul de difuzare a unei linii este: log P ($t_s + t_w N$)



a)



b)

Exemplu – Parallel Floyd 1 (3)

- Fiecare task este rădăcină pentru cel puţin o difuzare (dacă P ≤ N).
- Fiecare nod poate difuza în log P paşi
- Pentru N difuzări, cu mesaje de lungime N:

$$T_{Floyd1} = \frac{t_c N^3}{P} + N \log P \, (t_s + t_w N)$$

Timpul de comunicare

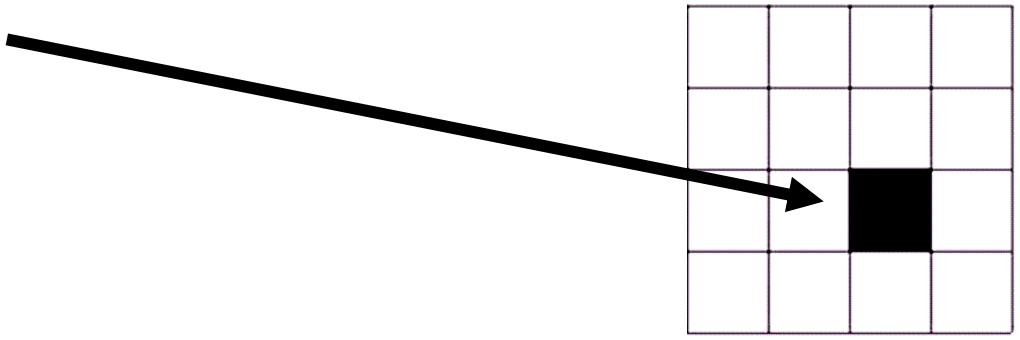
Timpul algoritmului secvențial împărțit la numărul de procesoare

Exemplu – Parallel Floyd 2 (1)

- Descompunere bi-dimensională a matricelor.
- Foloseşte până la N^2 procesoare.

```
for [k = 0 to N - 1]
  for [i = local_i_start to local_i_end]
    for [j = local_j_start to local_j_end]
      I[i,j]k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k)
```

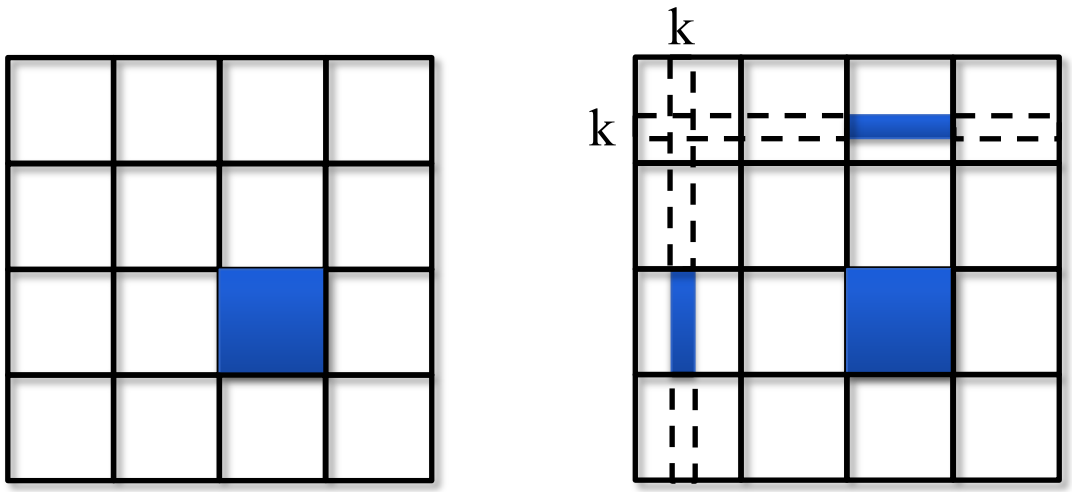
- Fiecare task răspunde de calculul elementelor situate intr-un patrat.



Exemplu – Parallel Floyd 2 (2)

- Cerinţe de comunicare în pasul k: două operaţii de difuzare.
- În fiecare pas, fiecare task cere, în plus faţă de datele locale, $\frac{N}{\sqrt{P}}$ valori de la două task-uri din aceeaşi linie şi aceeaşi coloană.
- Fiecare din cele doua mesaje are $\frac{N}{\sqrt{P}}$ elemente.
- Difuzarea unui mesaj durează (sunt $\sqrt{P}$ procese):

$$log\sqrt{P}\left(t_s + \frac{t_w N}{\sqrt{P}}\right)$$



Exemplu – Parallel Floyd 2 (3)

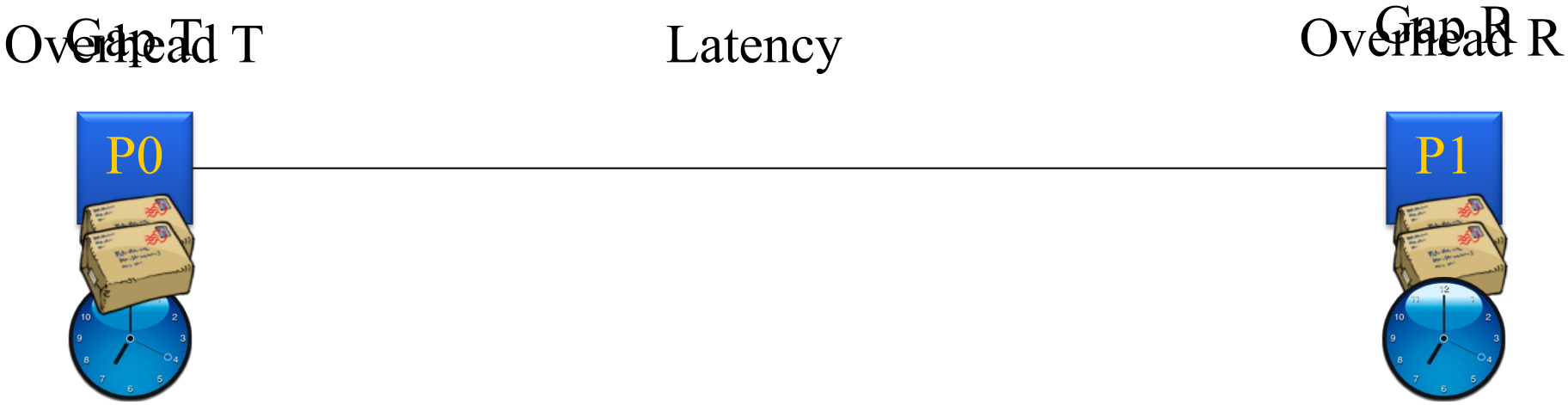
- În fiecare din N paşi, $\frac{N}{\sqrt{P}}$ valori trebuie difuzate la $\sqrt{P}$ task-uri în fiecare linie şi coloană; costul total este:

$$T_{Floyd\text{-}2} = t_c \frac{N^3}{P} + 2N \log\sqrt{P}\left(t_s + \frac{t_w N}{\sqrt{P}}\right)$$

$$= t_c \frac{N^3}{P} + N \log P \left(t_s + \frac{t_w N}{\sqrt{P}}\right)$$

Modelul LogP

Modelul LogP



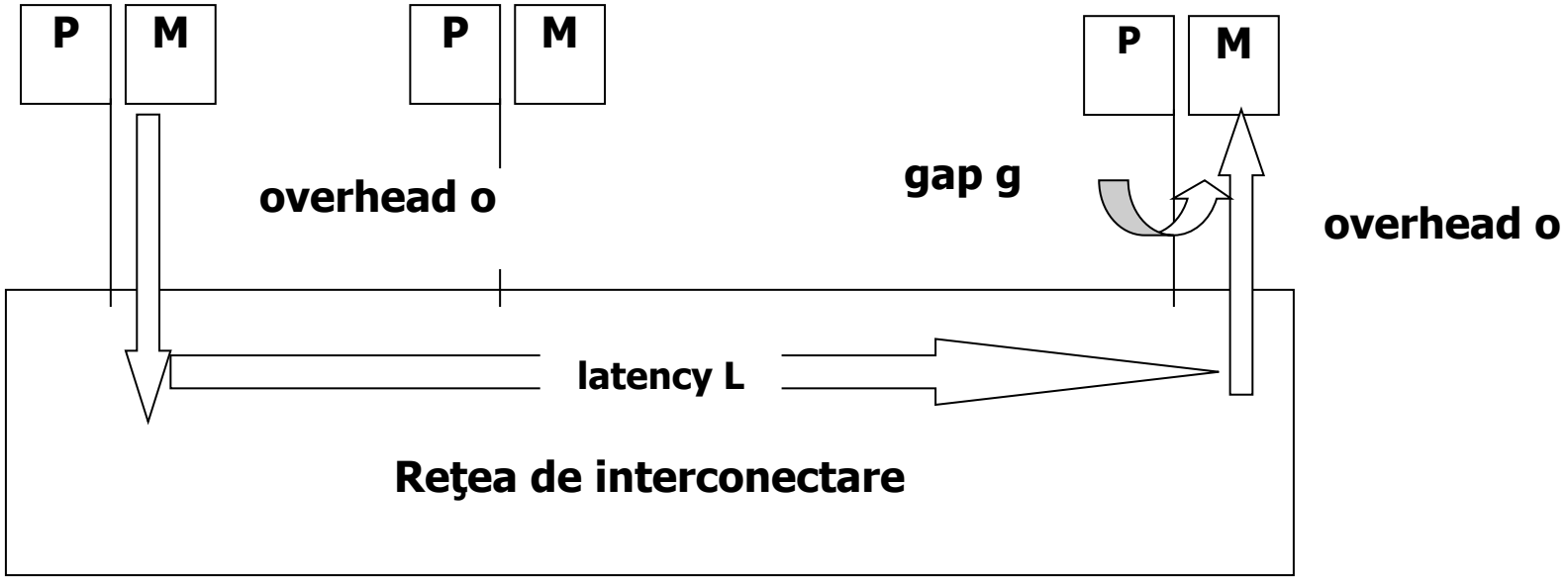
L - latency sau întârzierea de transmitere a unui mesaj mic de la sursă la destinatar

o - overhead, durata pentru care procesorul este angajat în transmiterea sau recepţia fiecărui mesaj

g - gap, intervalul minim de timp între două transmiteri succesive sau două recepţii succesive la acelaşi modul

P - numărul de module procesor / memorie.

Modelul LogP



L - latency sau întârzierea de transmitere a unui mesaj mic de la sursă la destinatar

o - overhead, durata pentru care procesorul este angajat în transmiterea sau recepţia fiecărui mesaj

g - gap, intervalul minim de timp între două transmiteri succesive sau două recepţii succesive la acelaşi modul

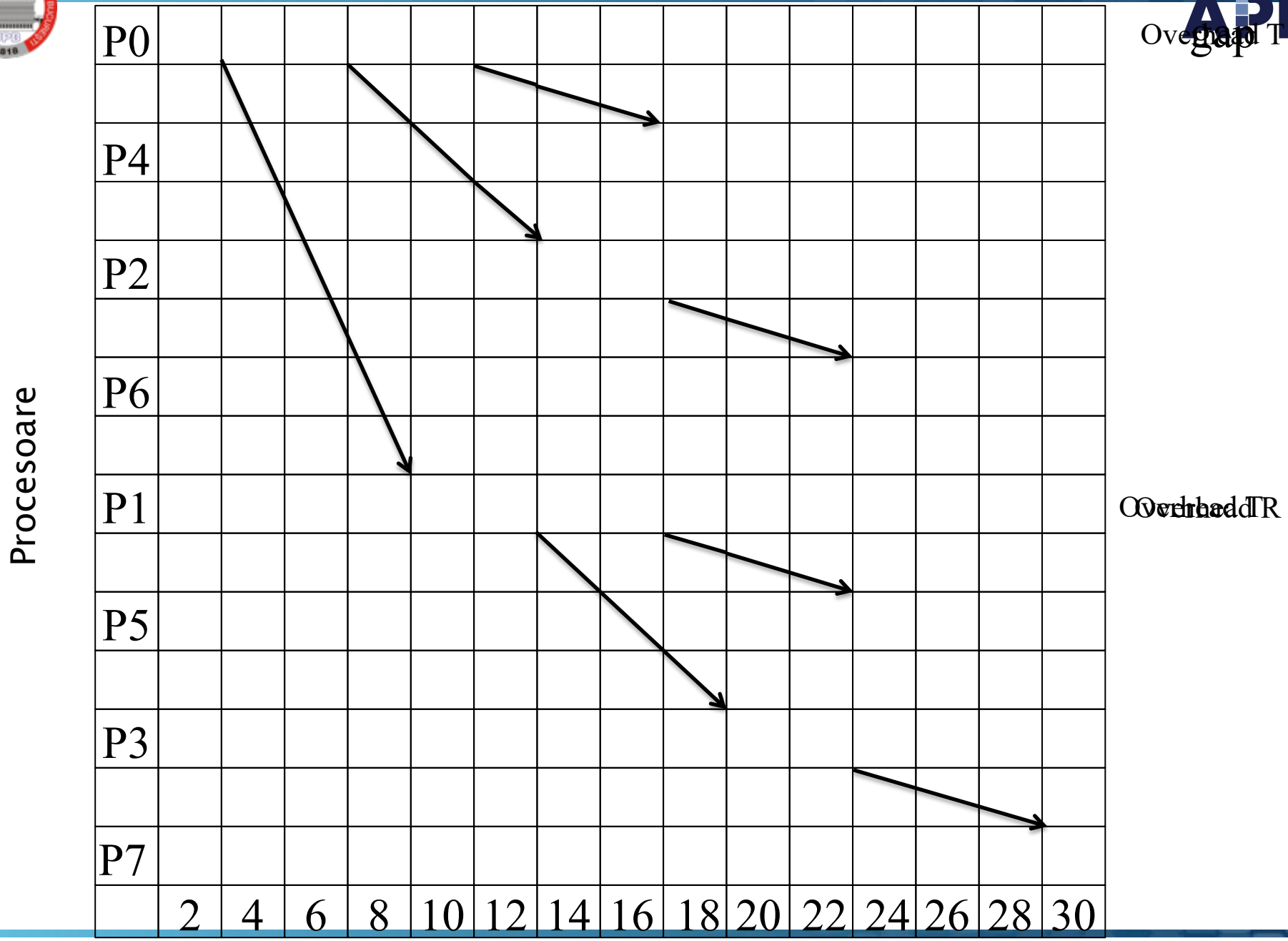
P - numărul de module procesor / memorie.

Exemplu - Difuzarea unei valori (1)

- Folosind modelului PRAM*, transmiterea are loc conform tiparului:
  - P0 → P1
  - P0, P1 → P2, P3
  - P0, P1, P2, P3 → P4, P5, P6, P7

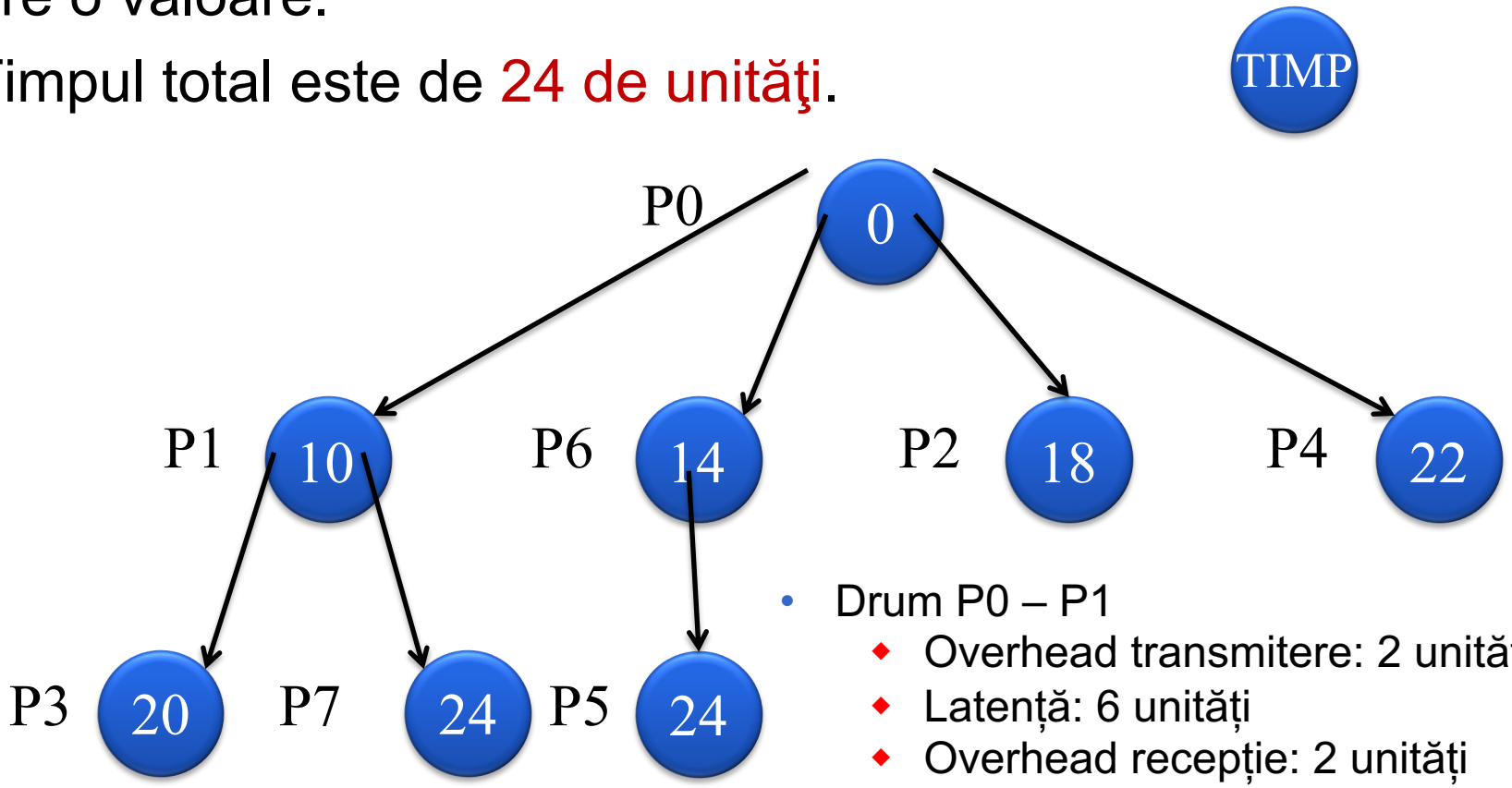
- Presupunând g ≥ 0 şi luând P = 8, o =2, g = 2, L = 6, obţinem timp total de difuzare = 30 de unităţi.

* Parallel Random Access Machine – vezi curs 1.

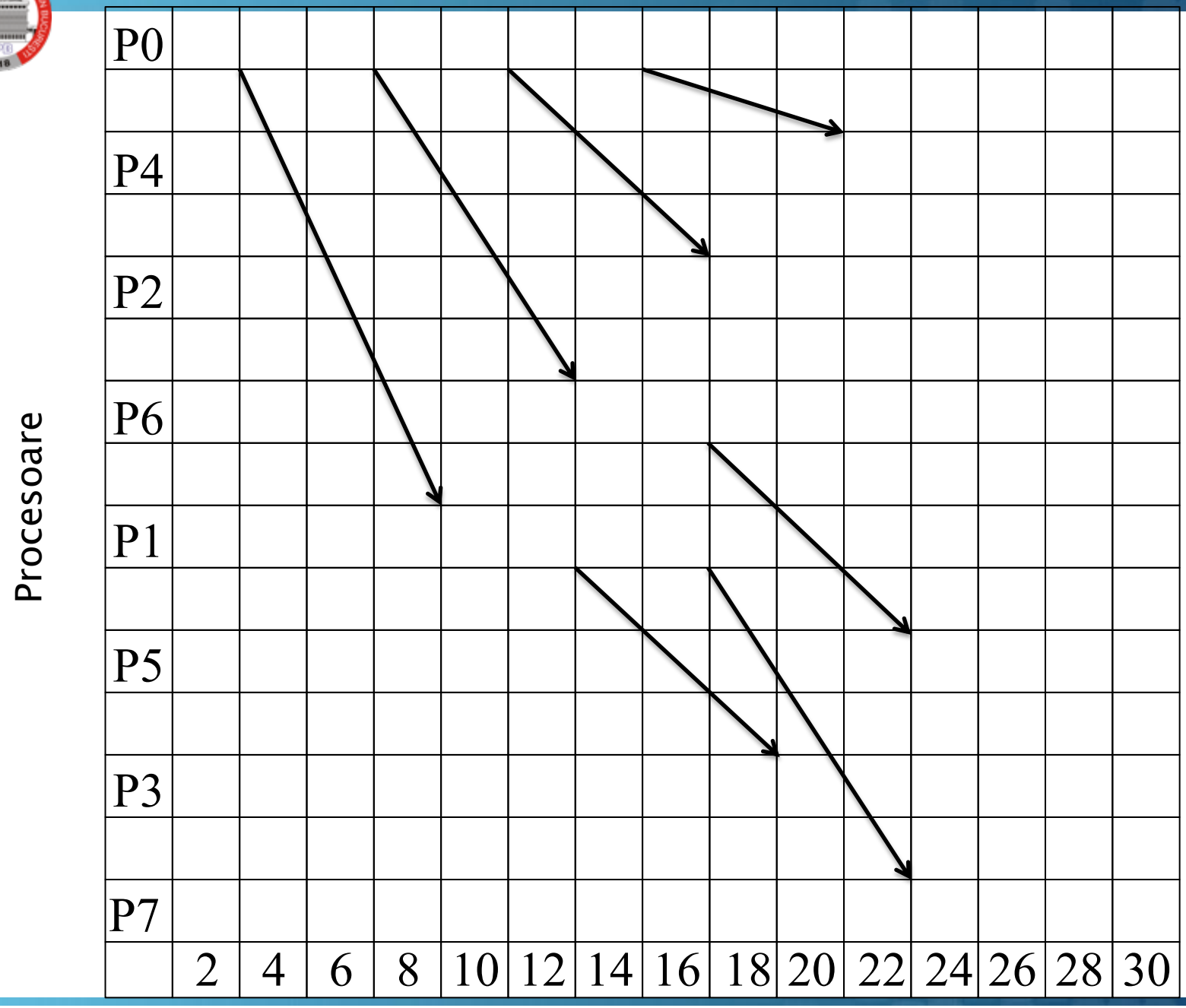

Overhead T
Overhead TR
Procesoare
P0
P4
P2
P6
P1
P5
P3
P7
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
Timp (unități de timp)

Exemplu - Difuzarea unei valori (2)

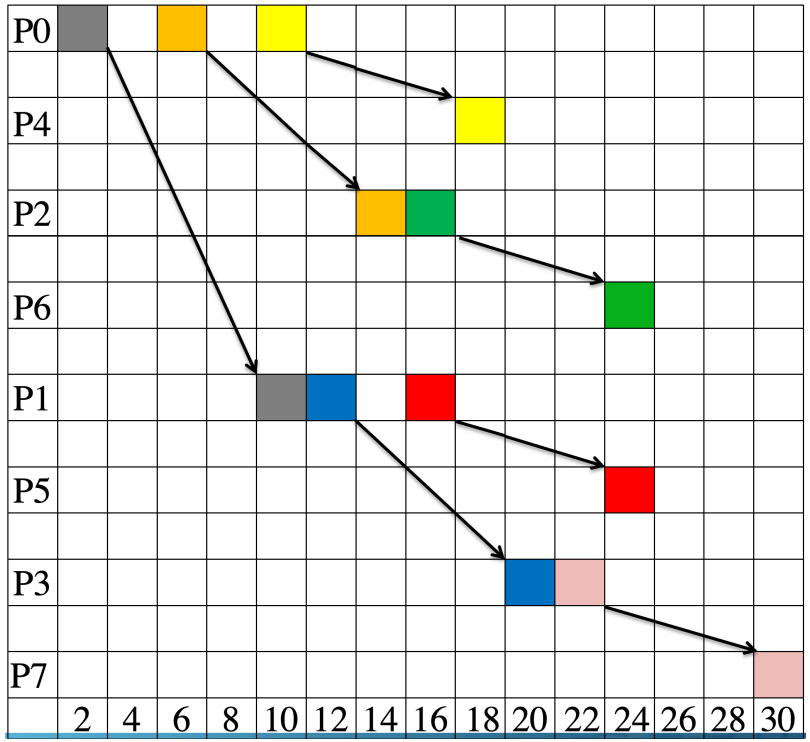
- Pentru modelul LogP, fiecare proces transmite imediat ce are o valoare.
- Timpul total este de 24 de unităţi.

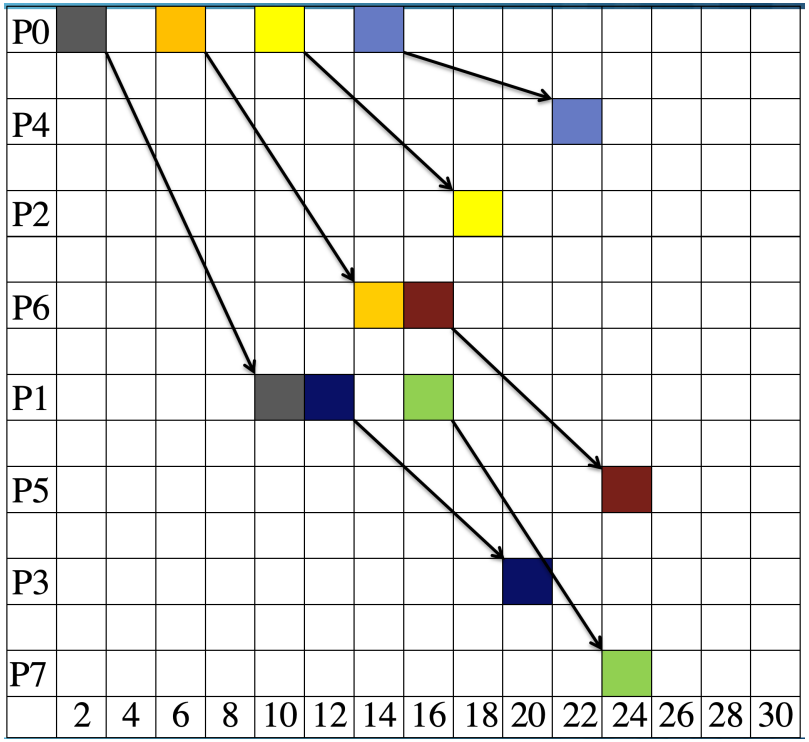


- Drum P0 – P1
  - Overhead transmitere: 2 unități
  - Latență: 6 unități
  - Overhead recepție: 2 unități
  - TOTAL: 10 unități


P0
P4
P2
P6
P1
P5
P3
P7
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
Procesoare
Timp (unități de timp)

Comparație


P0
P4
P2
P6
P1
P5
P3
P7
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30


P0
P4
P2
P6
P1
P5
P3
P7
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30

Sumar

- Comunicarea prin mesaje
  - Comunicarea asincronă prin mesaje
  - Modelul Replicated Workers
  - Comunicarea sincronă prin mesaje
- Complexitatea algoritmilor distribuiți
  - Modelul Foster
  - Modelul LogP